



GRUP 1

2026

**DOCUMENTACIÓ
TÈCNICA – CAT
XUXEMONS**

Prepared By :

Grup 1

Prepared For :

Carlos.M, Marc.C, Eric.G

Índex

Índex.....	2
Explicació General del Docker Compose.....	3
Creació del Docker Compose.....	5
Interfície d'usuari.....	6
Instal·lació de la imatge d'Angular.....	7
Nom del Contenidor.....	7
Volum (interfaç).....	7
Ports (Frontend).....	7
Comandes (al davant).....	8
Red (Front).....	8
Depends_on (Interfaç).....	8
Backend.....	9
Instal·lació de la imatge de MySQL.....	10
Nom del Contenidor.....	11
Volum (backend).....	11
Ports (Backend).....	12
Variables d'Entorn (backend).....	12
Red (basedades).....	13
Depends_on (Backend).....	13
Comandes (backend).....	13
Base de dades.....	15
Instal·lació de la imatge de MySQL.....	16
Nom del Contenidor.....	16
Variables d' Entorn (basedades).....	16
Volum (basedades).....	17
Ports (basedades).....	17
Vermell (basedat).....	17
Revisió de salut.....	18
Volum.....	18
Xarxes.....	19
Ordre per executar DockerCompose.....	19
Comprovació de Contenidors i Xarxes.....	20
Comprovació Contenidors i Xarxes.....	20
Comprovació Imatges.....	20
Comprovació Volums.....	21
Comprovació Xarxes PortNavigator.....	22

Explicació General del Docker Compose

En aquest apartat explicarem com crear una docker-compose.yml per poder crear dins del docker compon els serveis del (Backend i Frontend) amb els seus contenidors, volums i les seves xarxes.

```
1  name: xuxemonpro
2
3  services:
4    # Frontend (Angular)
5    frontend:
6      image: node:20-alpine
7      container_name: xuxemon-frontend
8      volumes:
9        - ./frontend:/app
10     ports:
11       - "4200:4200"
12     command: sh -c "cd /app && npm install && npm start -- --host 0.0.0.0"
13     networks:
14       - red
15     depends_on:
16       backend:
17         condition: service_started
18
19   # Backend (Laravel)
20   backend:
21     build:
22       context: ./backend
23       dockerfile: Dockerfile
24     container_name: xuxemon-backend
25     volumes:
26       - ./backend:/var/www/html
27     ports:
28       - "8000:80"
29     environment:
30       DB_CONNECTION: mysql
31       DB_HOST: basedatos
32       DB_PORT: 3306
33       DB_DATABASE: xuxemonsbd
34       DB_USERNAME: dev_user
35       DB_PASSWORD: 1234
36     networks:
37       - red
38     depends_on:
39       basedatos:
40         condition: service_healthy
41     command: >
42       sh -c "
43         php artisan config:clear &&
44         php artisan jwt:secret --force &&
45         php artisan migrate --force &&
46         php artisan db:seed --force &&
47         apache2-foreground
48       "
```

```
50 # Base de Datos (MySQL)
51 basedatos:
52   image: mysql:8.0
53   container_name: mysql-db
54   environment:
55     MYSQL_DATABASE: xuxemonsbd
56     MYSQL_USER: dev_user
57     MYSQL_PASSWORD: 1234
58     MYSQL_ROOT_PASSWORD: root_password
59   volumes:
60     - mysql_data:/var/lib/mysql
61   ports:
62     - "3306:3306"
63   networks:
64     - red
65   healthcheck:
66     test:
67       [
68         "CMD",
69         "mysqladmin",
70         "ping",
71         "-h",
72         "localhost",
73         "-u",
74         "root",
75         "-proot_password",
76       ]
77     interval: 10s
78     timeout: 5s
79     retries: 5
80
81   volumes:
82     mysql_data:
83
84   networks:
85     red:
86     driver: bridge
```

Creació del Docker Compose

El nostre nom del Dock Compose és el següent que veiem a la imatge:

```
name: xuxemonpro
```

Després vénen els serveis és on definim els contenidors que en aquest cas són el frontend, backend i base de dades que són els següents que veiem:

També és important tenir en compte les tabulacions

```
services:
```

- frontend (Angular)
- backend (Laravel)
- base de dades (MySQL)



Interfície d'usuari

En aquest apartat veurem el codi en general de backend veurem com funciona cada pas en general de les backend, com veiem a la següent imatge:

```
# Frontend (Angular)
frontend:
  image: node:20-alpine
  container_name: xuxemon-frontend
  volumes:
    - ./frontend:/app
  ports:
    - "4200:4200"
  command: sh -c "cd /app && npm install && npm start -- --host 0.0.0.0"
  networks:
    - red
  depends_on:
    backend:
      condition: service_started
```



Instal·lació de la imatge d'Angular

El Docker Compose al Servei de ([Frontend](#)), primer descarrega la imatge oficial de node:20-alpine que seria la següent imatge que veiem:

```
image: node:20-alpine
```

Nom del Contenidor

Dins del Servei de frontend, després de la imatge tindrem el nom del contenidor que anomenarem, com veiem a la següent imatge:

```
container_name: xuxemon-frontend
```

Volum (interfaç)

Al frontend tenim un volum on muntem a la carpeta local (./frontend) dins del contenidor a (/app), això serveix perquè es reflecteixi a l'instant dins del contenidor sense necessitat de reconstruir-lo el contenidor de nou, com veiem a la següent imatge:

```
volumes:  
  - ./frontend:/app
```

Ports (Frontend)

Als [ports exposem](#) el port 4200 del nostre contenidor 4200. La seva funció principal és fer accessible els serveis intern d'un contenidor a un host o xarxes externes com veiem a la imatge següent:

```
ports:  
  - "4200:4200"
```

Comandes (al davant)

A l'ordre entra a la carpeta (app) per instal·lar les dependències d'angular, inicialitzant angular amb un host local en aquest cas com veiem a la imatge següent:

```
command: sh -c "cd /app && npm install && npm start -- --host 0.0.0.0"
```

Red (Front)

El servei de frontend el connectem a través de la xarxa que és un bridge per connectar-lo a altres serveis:

```
networks:  
  - red
```

Depends_on (Interfaç)

En aquest apartat el Docker esperarà que el contenidor del backend hagi arrencat. Per esperar que un servei com a backend estigui completament operatiu utilitzem el service_healthy, que això requereix un healthcheck que més endavant ho expliquem detalladament. Veurem a la següent imatge com veiem el depends_on:

```
depends_on:  
  backend:  
    condition: service_started
```


Backend

En aquest apartat veurem el codi en general de backend veurem com funciona cada pas en general de les backend, com veiem a la següent imatge:

```
19      # Backend (Laravel)
20      backend:
21          build:
22              context: ./backend
23              dockerfile: Dockerfile
24          container_name: xuxemon-backend
25          volumes:
26              - ./backend:/var/www/html
27          ports:
28              - "8000:80"
29          environment:
30              DB_CONNECTION: mysql
31              DB_HOST: basedatos
32              DB_PORT: 3306
33              DB_DATABASE: xuxemonsbd
34              DB_USERNAME: dev_user
35              DB_PASSWORD: 1234
36          networks:
37              - red
38          depends_on:
39              basedatos:
40                  condition: service_healthy
41          command: >
42              sh -c "
43                  composer install --no-interaction --prefer-dist &&
44                  php artisan config:clear &&
45                  php artisan jwt:secret --force &&
46                  php artisan migrate --force &&
47                  php artisan db:seed --force &&
48                  apache2-foreground
49              "
50
```

Instal·lació de la imatge de MySQL

El Docker Compose al Servei de ([backend](#)), en comptes d'utilitzar una imatge ja feta la construïm a partir del Dockerfile amb la imatge d'apache que són els apartats següents:

```
build:
  context: ./backend
  dockerfile: Dockerfile
```

- Dependències del sistema
- Extensions PHP de laravel
- Instal·lació del Docker Compose
- Habilitem el Mod_rewrite d'Apache : activa el mòdul de reescriptura d'URL d'Apache
- Configuració de l'Apache per a Laravel
- El Workdir: és una instrucció dins del Dockerfile que estableix el directori de treball actual per a les instruccions subsegüents (RUN, CMD, ENTRYPOINT, COPY, ADD) i defineix la carpeta predeterminada en iniciar el contenidor.
- Exposem el port 80

Al següent apartat veiem el que inclou dins del Dockerfile:

```
1  FROM php:8.3-apache
2
3  # Instalar dependencias del sistema
4  RUN apt-get update && apt-get install -y \
5      git \
6      curl \
7      libpng-dev \
8      libonig-dev \
9      libxml2-dev \
10     libzip-dev \
11     zip \
12     unzip \
13     && rm -rf /var/lib/apt/lists/*
14
15 # Instalar extensiones PHP para Laravel
16 RUN docker-php-ext-install \
17     pdo_mysql \
18     mbstring \
19     exif \
20     pcntl \
21     bcmath \
22     gd \
23     zip
```

```
25 # Instalar Composer
26 COPY --from=composer:latest /usr/bin/composer /usr/bin/composer
27
28 # Habilitar mod_rewrite de Apache
29 RUN a2enmod rewrite
30
31 # Configurar Apache para Laravel
32 RUN echo '<VirtualHost *:80>\n\
33     DocumentRoot /var/www/html/public\n\
34     <Directory /var/www/html/public>\n\
35         AllowOverride All\n\
36         Require all granted\n\
37     </Directory>\n\
38     ErrorLog ${APACHE_LOG_DIR}/error.log\n\
39     CustomLog ${APACHE_LOG_DIR}/access.log combined\n\
40 </VirtualHost>' > /etc/apache2/sites-available/000-default.conf
41
42 WORKDIR /var/www/html
43
44 EXPOSE 80
```

Nom del Contenedor

Dins del Servei de Backend, després de la imatge tindrem el nom del contenidor que anomenarem, com veiem a la següent imatge:

```
container_name: xuxemon-backend
```

Volum (backend)

Al frontend tenim un volum on muntem a la carpeta local (./backend) dins del contenidor a (/var/www/html), això serveix perquè es reflecteixi a l'instant dins del contenidor sense necessitat de reconstruir-lo el contenidor de nou, com veiem a la imatge següent:

```
volumes:
  - ./backend:/var/www/html
```

Ports (Backend)

Als `ports` exposem el port `8000` del nostre contenidor `80`. La seva funció principal és fer accessible els serveis intern d'un contenidor a un host o xarxes externes com veiem a la imatge següent:

```
ports:
  - "8000:80"
```

Variables d'Entorn (backend)

A la Base de dades tenim un apartat amb variables d'entorn per poder configurar la base de dades per primera vegada i per poder entrar i veure la base de dades amb les vostres taules creades, que cada variable d'entorn el que fa és els següents:

- **DB_CONNECTION:**

Tipus de Connexió (mysql)

- **DB_HOST:**

Adreça del servidor de la base de dades

- **DB_PORT:**

Port de la base de dades

- **DB_DATABASE:**

Nom de la base de dades (xuxemonsbd)

- **DB_USERNAME:**

Usuari base de dades (xuxemonsbd)

- **DB_PASSWORD:**

Contrasenya base de dades (xuxemonsbd)

```
environment:
  DB_CONNECTION: mysql
  DB_HOST: basedatos
  DB_PORT: 3306
  DB_DATABASE: xuxemonsbd
  DB_USERNAME: dev_user
  DB_PASSWORD: 1234
```

Red (basedades)

El servei de basedades el connectem a través de la xarxa que és un bridge per connectar-lo a altres serveis:

```
networks:  
  - red
```

Depends_on (Backend)

En aquest apartat el Docker esperarà que el contenidor de basedades estigui completament operatiu abans d'arrencar el backend. A diferència del frontend que fa servir `service_started`, aquí s'utilitza `service_healthy`, cosa que significa que Docker no només espera que MySQL hagi arrencat, sinó que a més exigeix que superi un healthcheck real, és a dir, una prova que confirma que la base de dades està a punt per acceptar connexions. Això és imprescindible perquè el backend necessita executar `php artisan migrate` per crear les taules, i si MySQL encara s'està inicialitzant en aquest moment, la connexió fallaria i el contenidor cauria.

```
depends_on:  
  basedatos:  
    condition: service_healthy
```

Comandes (backend)

En aquest apartat el Docker esperarà que el contenidor de basedades estigui completament operatiu abans d'arrencar el backend. A diferència del

El Docker Compose al servei de backend, una vegada que el contenidor està dret, executa una sèrie d'ordres en ordre estricte que preparen Laravel abans d'arrencar Apache:

- **composer install --no-interaction --prefer-dist:**

Instal·la totes les dependències PHP del projecte llegint el fitxer `composer.json`, equivalent al que seria un `npm install` a Node.

- **php artisan config:clear:**

neteja la memòria cau de configuració perquè Laravel llegeixi correctament les variables d'entorn que li hem passat a l'apartat `environment`, com el `host` o les credencials de la base de dades.

- **php artisan jwt:secret --force:**

genera una clau secreta aleatòria que es farà servir per signar i verificar els tokens JWT, que són els que gestionen l'autenticació dels usuaris.

- **php artisan migrate --force:**

crea totes les taules de la base de dades a partir dels fitxers de migració de Laravel. El --force és necessari perquè estem en un entorn que Laravel detecta com a producció i sense ell demanaria confirmació manual.

- **php artisan db:seed --force:**

omple les taules acabades de crear amb dades inicials definides als seeders, com a usuaris per defecte o dades de prova.

- **apache2-foreground:**

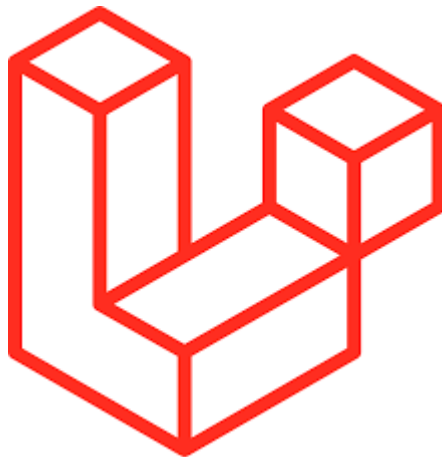
arrenca Apache en primer pla. Això és imprescindible perquè Docker mata un contenidor si el seu procés principal s'acaba, per la qual cosa Apache ha de mantenir-se actiu en primer pla perquè el contenidor segueixi viu.

```
command: >
  sh -c "
    composer install --no-interaction --prefer-dist &&
    php artisan config:clear &&
    php artisan jwt:secret --force &&
    php artisan migrate --force &&
    php artisan db:seed --force &&
    apache2-foreground
  "
```

Base de dades

En aquest apartat veurem el codi en general de base de dades veurem com funciona cada pas en general de les bases de dades, com veiem a la imatge següent:

```
51 # Base de Datos (MySQL)
52 basedatos:
53   image: mysql:8.0
54   container_name: mysql-db
55   environment:
56     MYSQL_DATABASE: xuxemonsbd
57     MYSQL_USER: dev_user
58     MYSQL_PASSWORD: 1234
59     MYSQL_ROOT_PASSWORD: root_password
60   volumes:
61     - mysql_data:/var/lib/mysql
62   ports:
63     - "3306:3306"
64   networks:
65     - red
66   healthcheck:
67     test:
68       [
69         "CMD",
70         "mysqladmin",
71         "ping",
72         "-h",
73         "localhost",
74         "-u",
75         "root",
76         "-proot_password",
77       ]
78     interval: 10s
79     timeout: 5s
80     retries: 5
```



Instal·lació de la imatge de MySQL

El Docker Compose al Servei de ([basedades](#)), primer descarrega la imatge oficial de MySQL 8.0 que seria la següent imatge que veiem:

```
image: mysql:8.0
```

Nom del Contenidor

Dins del Servei de base dades després de la imatge tindrem el nom del contenidor que anomenarem, com veiem a la imatge següent:

```
container_name: mysql-db
```

Variables d' Entorn (basedades)

A la Base de dades tenim un apartat amb variables d'entorn per poder configurar la base de dades per primera vegada i per poder entrar i veure la base de dades amb les vostres taules creades, que cada variable d'entorn el que fa és els següents:

- **MYSQL_DATABASE:**
Nom de la Base de Dades
- **MYSQL_USER:**
Usuari per iniciar a la base de dades
- **MYSQL_PASSWORD:**
Contrasenya de l'Usuari
- **MYSQL_ROOT_PASSWORD:**
Contrasenya del Root

```
environment:  
  MYSQL_DATABASE: xuxemonsbd  
  MYSQL_USER: dev_user  
  MYSQL_PASSWORD: 1234  
  MYSQL_ROOT_PASSWORD: root_password
```


Volum (basedades)

Al volum de Docker mysql_data a la carpeta de MySQL desa les seves dades físicament. Així quan fem un docker compose down les dades no s'esborren, que el volum és el següent que veiem a la imatge:

```
volumes:  
  - mysql_data:/var/lib/mysql
```

Ports (basedades)

Als [ports](#) [exposem](#) el port [3306](#) del nostre contenidor [3306](#). La seva funció principal és fer accessible els serveis intern d'un contenidor a un host o xarxes externes com veiem a la imatge següent:

```
ports:  
  - "3306:3306"
```

Vermell (basedat)

El servei de basedades el connectem a través de la xarxa que és un bridge per connectar-lo a altres serveis:

```
networks:  
  - red
```

Revisió de salut

El **Healthcheck** la funció que té a l'apartat de la base de dades és verificar que el servei de la Base de dades, té 5 intents de 10 segons cada intent, si falla 5 vegades seguides, passa a **insalubre** i no s'iniciarà MySQL.

A més el **Backend** té la (**condition: service_healthy**), amb aquest apartat no arrencarà fins que el **backend** es connecta a **MySQL**.

```
healthcheck:
  test:
    [
      "CMD",
      "mysqladmin",
      "ping",
      "-h",
      "localhost",
      "-u",
      "root",
      "-proot_password",
    ]
  interval: 10s
  timeout: 5s
  retries: 5
```

Volum

El Volum desa els registres en general i les migracions que incloem dins la base de dades.

En el cas que la Base de dades es destrueixi, reiniciï o es corrompi, així les dades que estiguin introduïdes es guarden a la següent ruta que veiem a la imatge, perquè així es conservin les dades si passa algun problema.

```
volumes:
  mysql_data:
```

Xarxes

La Xarxa, el que ens permet és que es connecti tots dos serveis que són el frontend i la base de dades una xarxa virtual privada, la qual cosa aquesta xarxa és un **pont** que serveix per connectar a través d'un pont tots dos serveis.

```
networks:  
  red:  
    driver: bridge
```

Ordre per executar DockerCompose

En el projecte de xuxemons haurem d'executar per primera vegada la següent ordre perquè es creïn les imatges, contenidors, volums, xarxes, que és el següent que veiem a la següent imatge:

```
$ docker compose up -d
```

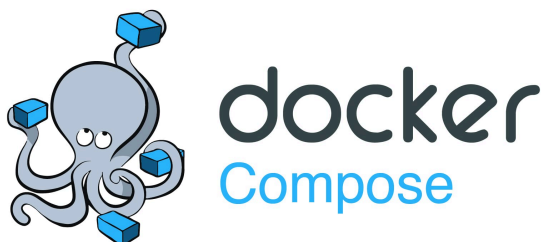
Amb la següent ordre eliminarem tots els serveis amb les seves respectives dades de la base de dades que és la següent ordre que veiem:

(el -v el que fa és esborrar les dades persistents i els volums associats)

```
$ docker compose down -v
```

A la següent ordre podrem utilitzar per poder-ho reconstruir tot

```
$ docker compose up -d --build
```



Comprovació de Contenidors i Xarxes

Al Docker Desktop comprovarem que s'hagin creat les imatges, contenidors, volums, xarxes etc... com veiem a les següents imatges:

Comprovació Contenidors i Xarxes

A la imatge com podem observar, podem veure que s'ha creat els contenidors amb les seves xarxes respectives cadascuna:

Containers [Give feedback](#)

Container CPU usage ⓘ 284.94% / 800% (8 CPUs available) Container memory usage ⓘ 714.4MB / 7.52GB [Show charts](#)

Search ☐ Only show running containers

☐		Name	Container ID	Image	Port(s)	CPU (...)	Last started	Actions
☐	☐	tender_wing	db0beacea21b	php:latest		0%	2 months ago	
☐	☒	xuxemonpro	-	-	-	229.6%	1 minute ago	
☐	☒	xuxemon-frontend	bb138e035f80	node:20-alpine	4200:4200 ↗	228.97%	1 minute ago	
☐	☒	xuxemon-backend	e26dd5590ecc	xuxemonpro-backend	8000:80 ↗	0%	1 minute ago	
☐	☒	mysql-db	024c01f8f3b5	mysql:8.0	3306:3306 ↗	0.63%	1 minute ago	

Comprovació Imatges

A la imatge com podem observar, podem veure que s'ha creat les imatges amb les seves xarxes respectives cadascuna:

Images [Give feedback](#)

Local My Hub

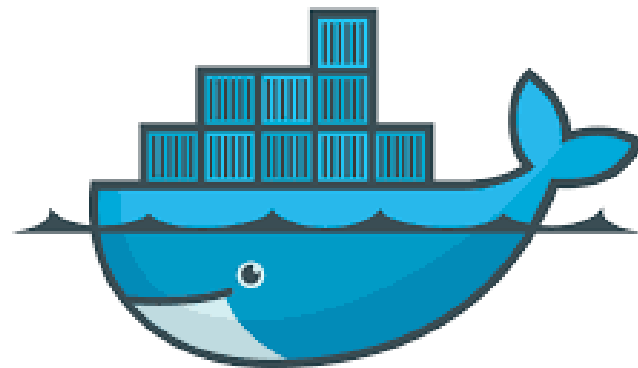
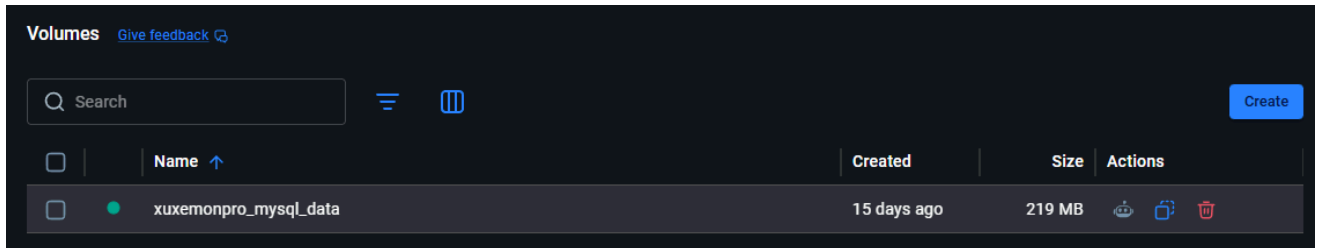
2.17 GB / 5.39 GB in use 5 images Last refresh: 21 hours ago ↻

Search

☐		Name	Tag	Image ID	Created	Size	Actions
☐	☒	xuxemonpro-backend	latest	d2104dab66f5	15 days ago	797.18 MB	
☐	☒	php	latest	7c99b9bd905e	2 months ago	795.64 MB	
☐	☒	mysql	8.0	99d774bf02a4	3 months ago	1.08 GB	
☐	☒	node	20-alpine	09e2b3d97260	3 months ago	192.04 MB	
☐	☐	portnavigator/port-navigator	1.1.0	1c3b1d792e15	3 years ago	235.45 MB	

Comprovació Volumes

A la imatge com podem observar, podem veure que s'han creat els volums amb les seves xarxes respectives:



docker



Comprovació Xarxes PortNavigator

A l'extensió del PortNavigator podem comprovar que els contenidors estan connectats a la mateixa xarxa mitjançant un pont que hi ha a la xarxa de Xuxemons.

The screenshot displays the PortNavigator Network interface for a network named 'xuxemonpro_red'. The interface shows the network configuration (Driver: bridge, Gateway: 172.18.0.1, Subnet: 172.18.0.0/16) and a list of three connected containers. Each container card displays its ID, image, activity, type, IPv4 address, published ports, and private ports, along with buttons to disconnect or connect to networks.

Container Name	ContainerID	Image	Activity	Type	IPv4Address	Published Ports	Private Ports
xuxemon-frontend	bb138e035f80508b9a101...	node:20-alpine	Up About a minute	tcp	172.18.0.4/16	4200:4200, 4200:4200	null
xuxemon-backend	e26dd5590ecc3782756ffb...	xuxemonpro-backend	Up About a minute	tcp	172.18.0.3/16	8000:80, 8000:80	null
mysql-db	024c01f8f3b5b128f2ca6d...	mysql:8.0	Up About a minute (healthy)	tcp	172.18.0.2/16	3306:3306, 3306:3306	null